

Neurolabs Cordova SDK Integration Guide

Technical Integration Specification

Document ID: NLB-CORDOVA-INTEGRATION-GUIDE

Version: v1.7.16

Date: 2026-10-09

Status: Partner integration reference

Owner: Neurolabs

1. Scope

This guide is for partner hybrid apps integrating 'neurolabs-cordova-sdk'.

2. Changes

The Cordova plugin is a thin bridge over the native SDKs, so most changes listed here come from the underlying 'neurolabs-android-sdk' and 'neurolabs-ios-sdk' releases that the plugin pulls via 'scripts/prepare-sdk.js' at install time. No public JS API has been broken since v1.1.9.

v1.7.16

- Plugin pins the native SDKs' 1.7.16 on both platforms.
- New 'init({ detectorPrefetchPolicy: 'unmetered' })': the detector downloads only on Wi-Fi or Ethernet (native SDK 1.7.16). While it waits, 'getDetectorModelStatus()' reports 'reason: "waiting_for_unmetered_network"'.
- New 'init({ recognition: false })': the plugin never builds on-device recognition.
- New section 6c, "Capture and quality checks only (no boxes, no recognition)", with an upgrade checklist from plugin 1.6.6.
- New 'openCamera({ finalReview: true })': an end-of-session review with tap-to-crop on the default camera (native SDK 1.7.16; see 6).
- New AR overlay options 'arBoxStyle', 'arBoxHighlightColor', 'arCoverageStyle', 'arCoverageOutlineColor', 'arBoxPersistence', and the experimental 'arLiveLabels' / 'arLiveCount' (native SDK 1.7.16; see 6b).
- New 'init({ maxRetainedPhotos, retainedPhotoExpiryDays })', the 'retainedPhotosHigh' event, 'retainedCount' / 'retainedBytes' in the queue status, and the 'STORAGE_LOW' capture refusal (native SDK 1.7.16; see 5 and 7). Uploaded photos kept on the device no longer count toward 'maxQueueSize'.
- Capture size: the plugin no longer sends 1920 px / 0.85 when you omit 'maxImageDimension' / 'imageCompressionQuality'. The SDK default is now 4032 px at JPEG 0.9 on both platforms, and Neurolabs can override it per organisation (see 6). Set the values yourself only if upload size matters more than recognition quality.
- 'validationPreset: "lenient"' now applies the native lenient thresholds (it used to apply the default ones), so fewer captures are refused.
- Native: quality checks run on the image alone until the detector is ready; the camera says why the detector is unavailable; uploaded photos never fill the upload queue; AR multi-bay handles right-to-left walks and shows the whole photo in the preview.

v1.7.15

- Plugin pins the native SDKs' 1.7.15 on both platforms.
- AR multi-bay ('preset: "multiBayAR"'): no per-bay preview; the bay is committed at the shutter and "Review shelf" after Finish is the one review (tap a bay to open it, crop, Previous / Next). Guidance holds while AR tracking is limited. New 'openCamera({ arFlowShowsCapturePreview: true })' brings the per-bay preview back (also needs 'showCapturePreview'; see 6b).
- iOS: the downloaded detector and embedder now install on a physical device (they never compiled on a phone with native 1.7.13 / 1.7.14).
- Facings and recognition shares count SKU boxes only, at or above the 0.5 match floor: counts may drop.

v1.7.14

- Plugin pins the native SDKs' 1.7.14 on both platforms. No JS API changes.
- 'detectorModelStatusChanged' / 'getDetectorModelStatus()' can report two more 'reason' codes: 'insufficient_storage' (free space, then the SDK retries straight away) and 'load_failed' (retried on the next launch).
- Native: on-device recognition keeps working offline from the last good sync; mission submits are idempotent; each tenant's upload queue stays separate across re-init.

v1.7.13

- Plugin pins the native SDKs' 1.7.13 on both platforms.
- The native SDKs bundle no detector: 'all_in_one' downloads on first use, so the first launch needs network for live detection (capture and uploads work without it). The package is about 100 MB smaller per platform.
- New 'getDetectorModelStatus()', 'prefetchDetectorModel()' and 'detectorModelStatusChanged', with one failure 'reason' vocabulary on both platforms.
- 'yoloModelVariant': 'all_in_one' is the only detector; the legacy names are deprecated aliases for it.
- Re-init: 'init' with another configuration now applies it, on a fresh native core. The effective operations key ('operationsApiKey', else 'apiKey') selects the upload queue, so each tenant's queued captures stay with that tenant. 'init' without 'operationsBaseUrl' resets the operations base to production instead of keeping a base set by an earlier 'init'.
- Boolean 'init' options are read the same way on both platforms: 'true'/'false', a number (0 is false), or "'true'/'false'", "'1'/'0'", "'yes'/'no'", "'on'/'off'". Android used to ignore numbers and the word forms. For example, 'uploadsEnabled: "no"' or '0' now disables uploads there.

v1.7.12

- Plugin pins the native SDKs' 1.7.12 on both platforms.
- iOS: distance guidance ("move closer / back off") works on the AR camera.
- Android: failed uploads retry on foreground and network return; the shelf overlay follows the camera.
- Bridge: argument checks are the same on both platforms (stricter rule); see the CHANGELOG for the behaviour changes.

v1.7.11

- Plugin pins the native SDKs' 1.7.11 on both platforms.
- Security (iOS bridge): the previous tenant's recognition provider is cleared on a credential change or a re-init without a key.
- A whitespace-only 'apiKey' is rejected on iOS, as on Android.
- Android: the shelf overlay and live boxes sit on the products in portrait.
- iOS: two crashes fixed (extreme remote-config values; repeated 'issuePriority' kinds).

v1.7.10

- Plugin pins the native SDKs' 1.7.10 on both platforms.
- Captures are portrait-only again: native 1.7.9's landscape orientation is switched off because the backend pipelines assume portrait images.
- 'openCamera' rejects a 'customCameraTemplate' that is not an object with 'INVALID_ARGUMENT'; one bad template field no longer discards the whole template on iOS.
- Android 'getPhoto' / 'deletePhoto' reject with 'INTERNAL_ERROR' instead of never settling when the queue read fails.

v1.7.9

- Plugin pins the native SDKs' 1.7.9 on both platforms.
- New 'openCamera' options for AR capture, forwarded on both platforms: 'captureMode: 'guided' | 'continuous' and the AR multi-bay preset ('preset: 'multiBayAR' or 'multiBayAR: {...}'). See section 6b.
- AR detection boxes are always world-locked. There is no renderer option: a leftover 'arBoxRenderer' key is ignored.
- New 'getARCapabilities()' returning '{ worldTrackingSupported, depthSupported, installRequired }'.
- Android: ARCore dependency 1.52.0 -> 1.56.0.

v1.3.2

- Plugin pulls native SDK v1.3.2 binaries on 'cordova plugin add': Android AAR + iOS xcframework refreshed via the mobile-dist manifests.
- Native v1.3.2 highlights (no JS surface change required):
 - Mixpanel ingestion now lands in the EU region (events were previously POSTed to the US endpoint and silently dropped). iOS also fixes a separate bug where Mixpanel rejected every event because 'time' was serialized as a string rather than a JSON number.
 - Done button stays responsive while the SDK persists captures - disk I/O moved off the main thread on both platforms, inline spinner / progress indicator surfaced.
 - Android: native 'SIGSEGV' fix in 'com.google.ai.edge.litert.Model.nativeLoadAsset' when the TFLite asset is stored compressed in the APK; the loader probes the asset with 'AssetManager.openFd' first and extracts to internal storage when the asset is not mmap-able.
 - Onboarding barcode pipeline normalises to GTIN-14 end-to-end and propagates the scanned barcode format through every product-lookup client.
 - iOS: new 'NLOperationsCatalogClient' (in 'ProductAuditKit') for the operations catalog; replaces 'precondition'-based validation with thrown errors so a malformed barcode no longer crashes the host app.

v1.2.5

- Native sequence-counter parity: Cordova bridge surfaces the 'showsSequenceCounterLabel' (and 'showsCapturedCountLabel') options to match the native SDK toggles.
- Demo SDK setup builds fixed; iOS Sentry injection timing hardened; Android LiteRT dependency pinning correction; sequence-counter argument plumbed through correctly.

v1.2.3

- 'openCamera' supports 'cameraMode: "ar" | "default"' - "'default'" enables the 0.5/1 lens switcher on both iOS and Android.
- 'init' supports 'analyticsEnabled', 'sentryEnabled', 'sentryDsn' for controlling SDK analytics and crash reporting.
- Tightened default quality validation thresholds (blur, glare, perspective) across both platforms.

v1.2.2

- Internal: Android plugin integration handling refactor; no public JS API change.

v1.2.1

- Cordova install hooks hardened; demo integration updated.

v1.2.0

- Internal scaffolding for the v1.2.x line; no JS API change.

v1.1.9

- Init and per-session routing use 'taskUUID'.
- Native detector/model warmup is enforced in 'init' and guarded in 'openCamera'.
- 'openCamera' can return 'MODEL_INIT_FAILED' when native model initialization fails.
- 'autoCloseAfterCapture=true' now keeps preview enabled and closes after Save confirmation.
- Manual finish ('Done') is available when 'autoCloseAfterCapture=false' and 'allowManualFinish=true'.
- 'cameraClosed' event now includes 'message'.
- 'captureQueued' events are deferred while camera is open and flushed after 'cameraClosed'.

3. Install

Download the plugin package from <https://github.com/neurolaboratories/neurolabs-mobile-dist/releases>. The install hooks download and wire up the native SDKs automatically - no manual AAR or xcframework placement needed.

Native baseline requirements:

- Android: 'minSdkVersion 26', 'compileSdkVersion 36', JDK 17, Android Gradle Plugin 8.9.1+ (the SDK's androidx dependencies require it). The plugin injects 'GradlePluginKotlinVersion 2.3.0' and raises the Java source/target compatibility to 17 on its own; override the Kotlin version only via a '<platform name="android">'-'scoped preference.

- iOS: deployment target iOS 17.0+, Xcode 16.4+ recommended (release toolchain baseline).

Add to 'config.xml' (cordova-android 14 defaults are lower):

```
<platform name="android">
  <preference name="android-minSdkVersion" value="26" />
  <preference name="android-compileSdkVersion" value="36" />
  <preference name="AndroidGradlePluginVersion" value="8.9.1" />
  <preference name="GradleVersion" value="8.13" />
</platform>
```

Android (Windows or macOS)

```
# Add the Android platform first, then install the plugin
cordova platform add android
cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

The 'before_plugin_install' hook downloads 'neurolabs-android-sdk.aar' from the matching GitHub release, stores it inside the plugin, and 'plugin.xml' copies it to 'app/libs/' automatically. 'neurolabs.gradle' wires it into the build via 'flatDir'.

iOS (macOS only)

```
# Add the iOS platform first, then install the plugin
cordova platform add ios
cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

The hook downloads 'NeurolabsSDK.xcframework' from the matching GitHub release and Cordova embeds it automatically during build. Requires 'curl' (standard on macOS) or 'gh' CLI. For private releases set 'GITHUB_TOKEN'.

The iOS install hook injects the 'Sentry' Swift Package dependency into the generated Xcode project automatically during platform preparation. If Xcode still reports 'unable to find module dependency: Sentry', re-run 'cordova prepare ios' so the package graph refreshes in the generated project.

Optional overrides (use a pre-downloaded file or a specific URL instead of auto-download):

```
# Local file
NEUROLABS_IOS_XCFRAMEWORK_ZIP=/path/to/NeurolabsSDK.xcframework-vX.Y.Z.zip \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

NEUROLABS_ANDROID_AAR_PATH=/path/to/neurolabs-android-sdk-vX.Y.Z.aar \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

# Direct URL
NEUROLABS_IOS_XCFRAMEWORK_URL=https://github.com/.../NeurolabsSDK.xcframework-vX.Y.Z.zip \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

NEUROLABS_ANDROID_AAR_URL=https://github.com/.../neurolabs-android-sdk-vX.Y.Z.aar \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

iOS + Android (macOS)

Add both platforms before installing the plugin - both hooks run in a single 'cordova plugin add'.

4. SDK Initialization + Warmup

```
const Neurolabs = cordova.require('ai.neurolabs.cordova.Neurolabs');

await Neurolabs.init({
  apiKey: '<API_KEY>',
  operationsApiKey: '<OPERATIONS_API_KEY>', // REQUIRED for uploads (see note below)
  // operationsBaseUrl: 'https://api.operations.staging.neurolabs.ai/v1', // optional override (staging / self-hosted)
  taskUUID: '<DEFAULT_TASK_UUID>',
  allowBase64PhotoExport: false,
  returnCaptureFileUri: false, // v1.6.5 opt-in file-URI streaming (see 8.1)
  analyticsEnabled: true, // SDK analytics (default true)
  sentryEnabled: true, // Sentry crash reporting (default true)
  // maxRetainedPhotos: 500, // kept uploaded photos before retainedPhotosHigh warns (see 5)
  // retainedPhotoExpiryDays: 14, // delete kept uploaded photos after N days (default off)
  // sentryDsn: 'https://...', // optional custom Sentry DSN
  // yoloModelVariant: 'all_in_one', // the only detector; see below
});
```

> **Upgrade note (pre-1.6 integrators): 'operationsApiKey' is now required for uploads.**

> > The legacy '/images' upload endpoint has been removed - the native > capture queue is operations-only. If you call 'init()' **without** > 'operationsApiKey', 'init()' and 'openCamera()' still succeed and photos > still queue, but **every upload fails silently**: the queue item errors > with 'OPERATIONS_KEY_REQUIRED' (non-retryable), surfaced only through the > 'uploadFailed' event. There is no console error and no init-time > rejection, so this is easy to miss during an upgrade. > > ""js > Neurolabs.addListener('uploadFailed', ({ errorCode, message }) => { > if (errorCode === 'OPERATIONS_KEY_REQUIRED') { > console.error('Missing operationsApiKey in init() - uploads will never succeed.', message); > } > }); > "" > > 'operationsBaseUrl' (optional) overrides the operations endpoint for > staging / self-hosted deployments. It must be an 'https' URL with a host > and no embedded credentials; 'http', malformed, or userinfo-bearing > values reject at 'init()' with 'INVALID_ARGUMENT'. Omit it to keep the > production default ('https://api.operations.neurolabs.ai/v1').

Detector model (downloaded at runtime)

Since native SDK 1.7.13 the SDK bundles **no** detector model. Its one detector, 'all_in_one', is downloaded on first use through the Operations API with the same operations credential as uploads, verified against a pinned sha256 and cached on the device. The plugin package is about 100 MB smaller per platform as a result.

- **The first launch needs network for detection.** Until the model is on the device, capture, the shutter, the upload queue and the pixel quality checks (light, glare, blur) all work; live detection, the product overlay and detection-based guidance do not, and the native camera shows a "Detector downloading. You can still capture." chip. When the download lands, detection switches on mid-session with no restart.
- **Loading progress.** Without the model the SDK reaches 'loadingProgress.step === "ready"' ("Ready (detector downloading...)") and never emits "'model_ready'". 'waitUntilReady()' resolves on "'ready'"; never wait for "'model_ready'" alone.

- **Status.** `Neurolabs.getDetectorModelStatus()` resolves `{ state, ready, reason? }` (`'state': "not_downloaded" | "downloading" | "ready" | "failed"`), and `'detectorModelStatusChanged'` fires on every change. `'Neurolabs.prefetchDetectorModel()'` (after `'init'`) starts the download now, e.g. on Wi-Fi before the first capture; a failed download resolves with `'state: "failed"'` and a `'reason'`, it does not reject. `'reason'` is one of `'no_credentials'`, `'not_entitled'`, `'unknown_model'`, `'pin_mismatch'`, `'sha_mismatch'`, `'download_failed'`, `'api_unavailable'`, `'compile_failed'`, `'insufficient_storage'`, `'load_failed'` on both platforms; anything else arrives as `'download_failed'`. The first five are retried after 1 h, the next three after 60 s; `'insufficient_storage'` (native SDK 1.7.14) is retried with no wait once space is freed (next capture session or `'prefetchDetectorModel()'`), and `'load_failed'` (1.7.14) only after the app is relaunched.
- **Downloading only on Wi-Fi.** `'init({ detectorPrefetchPolicy: 'unmetered' })'` (native SDK 1.7.16) holds the SDK's own download until the device is on an unmetered network. Meanwhile the status is `'not_downloaded'` with `'reason: "waiting_for_unmetered_network"'`, which is not a failure, and `'prefetchDetectorModel()'` still downloads at once. See 6c.
- **'DETECTOR_UNAVAILABLE'** (detection before the model lands) is expected and retryable. The plugin never reports it as `'uploadFailed'` or as an error.

`'yoloModelVariant'` (optional) is now effectively fixed: its default and only value is `'all_in_one'`. The legacy values `'nlb_model'`, `'yolo_v26'`, `'yolo_v11_seg'`, `'shelf_rows'` and `'yolo11l_seg_sam'` are **deprecated** aliases of `'all_in_one'`: still accepted, so existing configs keep working, but they no longer select a different model. An unknown value raises `'INVALID_ARGUMENT'`.

Warmup is native-side; monitor progress through `'loadingProgress'` event.

```
Neurolabs.addListener('loadingProgress', (payload) => {
  console.log('loading', payload);
});
```

5. Queue Management

```
await Neurolabs.setAutoSyncEnabled(true);
await Neurolabs.setWifiOnlyUploadsEnabled(false);
await Neurolabs.setDeletePhotosOnUploadSuccess(true);

await Neurolabs.pauseQueue();
await Neurolabs.resumeQueue();
await Neurolabs.setRetryPolicy({ retryCount: 5 });

const status = await Neurolabs.getQueueStatus();
console.log('queue status', status);

await Neurolabs.flushQueue();
await Neurolabs.retryFailedUploads();
```

Kept uploaded photos (native SDK 1.7.16)

With 'deletePhotosOnUploadSuccess: false' (keep-photos mode), or while a session waits for its submit, the queue keeps uploaded photos on the device. From native SDK 1.7.16 they take **no** 'maxQueueSize' slot: only pending, uploading and failed items do, so a host clean-up that lags no longer turns every new capture into 'QUEUE_FULL'.

- 'init({ maxRetainedPhotos })' (whole number > 0, default 500) bounds them with a warning only. From 80% of it the 'retainedPhotosHigh' event fires with '{ count, bytes, oldestCreatedAt?, limit }' ('oldestCreatedAt' in epoch ms), once per crossing and again only after the count drops back below. It never refuses a capture.
- 'init({ retainedPhotoExpiryDays })' (whole number > 0, or 'null' / omitted for off, the default) deletes a kept photo, files included, captured at least that many days ago, at the next flush or launch. Photos kept only until their session's submit never expire.
- Both are process-wide on the native side, so every 'init' sets them; omitting one restores its default. Zero, a negative, a fraction, a boolean or a string rejects with 'INVALID_ARGUMENT'.
- 'getQueueStatus()' and 'queueStatusChanged' carry 'retainedCount' and 'retainedBytes'.
- A new capture is refused with 'STORAGE_LOW' (retryable) when the queue's storage has under 500 MB free; it wins over 'QUEUE_FULL'. Both arrive through 'uploadFailed', the same way on iOS and Android (see 7).

```
await Neurolabs.init({ apiKey: '<API_KEY>', deletePhotosOnUploadSuccess: false, maxRetainedPhotos: 300 });

Neurolabs.addListener('retainedPhotosHigh', async ({ count, bytes, limit }) => {
  console.warn('Kept ${count}/${limit} uploaded photos (${bytes} bytes); cleaning up');
  await cleanUpHostCopies(); // e.g. Neurolabs.deletePhoto({ queueItemId }) for the oldest ones
});
```

6. Open Custom Camera

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  cameraMode: 'default',           // 'ar' or 'default' - enables 0.5/1 lens switcher
  guidanceMode: 'strict',
  validationPreset: 'ios_parity',

  confidenceThreshold: 0.25,
  iouThreshold: 0.45,
  maxCaptures: 1,

  showAlignmentGuidance: true,
  enableValidation: true,
  liveQualityChecksEnabled: true,
  liveQualityTargetFps: 6,

  // keep custom-camera guidance UI path
  showDetections: false,
  showCapturedRegions: false,

  // optional metadata/cropping
  sendDetectionsMetadata: true,
  enablePreviewCropping: true,

  // now defaults to true if omitted, but explicit is clearer
  autoCloseAfterCapture: true,

  // manual-finish mode (Done button):
  // allowManualFinish: true,
  // minCapturesBeforeDone: 3,

  // per-session routing override
  taskUUID: 'bfc85982-b955-4f65-9f32-b6dbed85f364',

  // image size and quality: omit them unless upload size matters more than
  // recognition accuracy (see "Image size and quality" below)
  // maxImageDimension: 4032,
  // imageCompressionQuality: 0.9
});
```

End-of-session review ('finalReview', native SDK 1.7.16)

A host that opens the default camera with 'showCapturePreview: false' had no review at all: a rep could not crop or drop a bad shot. 'finalReview: true' adds one review at the end of the session:

```
await Neurolabs.openCamera({
  cameraMode: 'default',
  showCapturePreview: false,
  autoCloseAfterCapture: false,
  allowManualFinish: true,
  finalReview: true,
});
```

- It applies to the default (non-AR) camera with no 'multiBay'; it is ignored on the AR camera. There is no per-shot preview, whatever 'showCapturePreview' says.

- Done, or the count that ends the session today ('maxCaptures', the auto-close target, or the required count when there is no Done button), opens **Review** with every photo. Tapping one opens it with Crop, Delete, Previous / Next and "Back to review".
- A crop replaces the photo under the same 'captureId'. A delete fires 'captureDeleted' with 'source: "review"'.
- Nothing from the session is saved or queued before **Confirm**, so 'captureQueued' arrives only after it. **Back to camera** keeps the photos and the shutter takes more; is the camera's own close.

Image size and quality

'maxImageDimension' (long edge, px), 'maxImageWidth', 'maxImageHeight' (px, '> 0') and 'imageCompressionQuality' (JPEG, '0..1') are forwarded only when you set them; the plugin has no default of its own. The native SDK resolves each one in this order:

1. the value you pass to 'openCamera';
2. your organisation's remote config ('sdkCapture.maxImageDimension' / 'sdkCapture.imageCompressionQuality' in 'GET /v1/recognition-config', set by Neurolabs per organisation);
3. the SDK default: a **4032 px long edge at JPEG quality 0.9** (native SDK 1.7.16).

The plugin pins native 1.7.16, so on both platforms an omitted value means 4032 px at 0.9 unless your organisation overrides it. Invalid values ('<= 0', or a quality outside '0..1') reject with 'INVALID_ARGUMENT' on both platforms.

Lowering them costs recognition quality. The product crops, the price matcher and OCR all work on the uploaded photo, not on the live preview. A smaller or more compressed photo leaves fewer pixels in each product crop and price tag, so matching and price reading get worse. Set them only when upload size matters more than recognition accuracy.

Upgrade note. Earlier plugin releases sent '1920' / '0.85' whenever you omitted these options. The SDK read those as your own values, so they overrode both your organisation's config and the SDK default. If you relied on that, you now get larger photos (4032 px at 0.9 with the native 1.7.16 pins) unless you set the values yourself or your organisation overrides them.

Manual-finish example:

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  guidanceMode: 'strict',
  maxCaptures: 10,
  autoCloseAfterCapture: false,
  allowManualFinish: true,
  minCapturesBeforeDone: 3
});
```

6a. Multi-Bay Capture (wide shelves)

Multi-bay capture (native SDK v1.6.0+) guides the user across several overlapping "bays" of a wide shelf in a single session and de-duplicates detections across them. Enable it by passing a 'multiBay' options object to 'openCamera':

```

await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  guidanceMode: 'strict',
  multiBay: {
    minBays: 2,           // 1..8
    maxBays: 4,           // 2..8
    targetOverlapFraction: 0.30, // 0.05..0.9 (native default 0.30)
    onDeviceDedup: true,    // de-duplicate detections across bays on device
    generatePreview: true,   // build a stitched display-only preview
    previewMaxDimension: 2048 // max long-edge px of the stitched preview
  }
});

```

Out-of-range values reject with 'INVALID_ARGUMENT', as does 'minBays > maxBays'. The ceiling is 8 bays in every mode, on both platforms: native SDK 1.7.16 lets a continuous AR sweep plan up to 30 bays, but only for an in-process host; the plugin, like the native bridge types, keeps 8. An optional 'guidanceStyle: 'classic' | 'glance"' (case-insensitive) selects the guidance overlay style.

Attach free-text submit notes to the session before its payload uploads:

```

await Neurolabs.setMultiBaySubmitNotes('Aisle 4, promo end-cap', sessionId);

```

When a multi-bay session completes, the 'cameraClosed' payload carries an extra 'multiBay' summary (forwarded untouched from the native SDK). The per-photo 'captureQueued' / upload events are unchanged:

```

Neurolabs.addListener('cameraClosed', ({ sessionId, cancelled, captureCount, message, multiBay }) => {
  if (!multiBay) return; // single-bay session
  const {
    countsByLabel, // { [label]: count } de-duplicated across bays
    baysCount,     // number of bays captured
    dedupTrusted,  // false if the cross-bay alignment chain broke
    previewImageFileUri // 'file:///...' when generatePreview: true (optional)
  } = multiBay;
  console.log('multi-bay summary', baysCount, dedupTrusted, countsByLabel);
});

```

'dedupTrusted: false' means the on-device de-duplication could not be fully trusted (the cross-bay alignment chain broke, the dedup timed out, or 'onDeviceDedup' was disabled) - counts are then per-bay sums (an upper bound). Treat 'countsByLabel' as an over-count in that case: the backend serves the raw per-image results unchanged (no server-side re-deduplication exists today - the uploaded alignment homographies make one possible later, but on-device dedup is currently the only cross-bay dedup).

There is no partner-facing in-camera bay-mode switch: the Single Multi-bay toggle shown in Neurolabs demo builds is internal debug chrome (gated behind 'NLSDKDebug.internalCameraToolsEnabled' on iOS and its Android equivalent, never enabled in partner builds). The capture mode is controlled purely by config: pass 'multiBay' for a multi-bay session, omit it for single-bay.

6b. AR Multi-Bay Capture (native SDK v1.7.8+)

On the AR camera ('cameraMode: 'ar') one more option applies:

- 'captureMode: 'guided' | 'continuous' (case-insensitive, default "guided"). Guided keeps the shutter and shows side guidelines for the next frame; continuous captures on its own as the rep moves along the shelf, with a higher battery and thermal cost. Without a pose (a non-AR camera, or Android without Google Play Services for AR) the camera keeps the shutter and behaves as guided.

AR detection boxes are always world-locked: they lie on the shelf plane and stay put as the rep walks. There is no renderer option. 'arBoxRenderer' was removed with the native legacy renderer, and a leftover 'arBoxRenderer' key from older code is ignored on both platforms, not rejected.

AR overlay style (native SDK 1.7.16)

Opt-in looks for demos and partners; the rep defaults are unchanged. The values are the native SDKs' wire values, case-insensitive. An unknown value is ignored and the default applies (it is never rejected), as the native SDKs decode it. They apply on the AR camera, with or without the preset.

- 'arBoxStyle': "quiet" (default; alias "standard"), "identity" (a colour per product) or "highlight" (every box a solid frame in 'arBoxHighlightColor', default '#22C55E', slightly thicker, no fill).
- 'arCoverageStyle': "tint" (default), "outline" (a thin line in 'arCoverageOutlineColor', default '#FACC15', drawn on the shelf plane around the captured bays) or "tintAndOutline" (alias "tint_and_outline"). With no shelf-plane fit yet every style shows the tint.
- 'arBoxPersistence': "standard" (default: a box fades about 18 s after it leaves view) or "session" (a settled box stays, dimmed, for the rest of the session; at most 300). Presentation only: counts are unchanged.
- Colours are '#RRGGBB' strings, forwarded as given; a malformed value draws the default colour.

AR live labels and count (experimental, native SDK 1.7.16)

- 'arLiveLabels: true' shows the top-1 product name on each stable box, at a match score of 0.6 or more, from the on-device recognition provider. With no provider (recognition off or not ready) there are no labels.
- 'arLiveCount: true' shows a "~ N products" chip: the distinct boxes seen stable this session. It is an estimate; the review keeps the exact count.
- The native 'productDetailsSource' (the catalogue resolver behind product details) is an in-process object and is **not exposed through Cordova**. Use 'getProductDetails(ids)' instead.

```
await Neurolabs.openCamera({
  preset: 'multiBayAR',
  arBoxStyle: 'highlight',
  arCoverageStyle: 'tintAndOutline',
  arBoxPersistence: 'session',
  arLiveLabels: true,
  arLiveCount: true,
});
```

For AR multi-bay, use the preset rather than assembling the options by hand:

```
// Guided (default): shutter + side guidelines, 1..4 bays
await Neurolabs.openCamera({ sessionId: crypto.randomUUID(), preset: 'multiBayAR' });

// Same preset with its inputs nested, here continuous and up to 6 bays
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  multiBayAR: {
    captureMode: 'continuous',
    multiBay: { minBays: 1, maxBays: 6 }
  }
});
```

The preset is the native SDKs' own 'NLNativeCaptureConfig.multiBayAR(...)' factory: AR camera, multi-bay on (1..4 bays unless you pass 'multiBay'), one review at the end, manual finish instead of auto-close, alignment guidance, glance guidance and detections metadata. Passing 'cameraMode: 'ar' + 'multiBay' yourself does NOT reproduce it, because the plain 'openCamera' defaults (auto-close on, preview on) still apply.

How the AR multi-bay flow reviews bays (native SDK 1.7.15+):

- There is no per-bay "Snapshot preview". Each bay is committed at the shutter (a flash, a toast and a thumbnail), and the AR session keeps running between bays, so tracking and guidance carry on.
- "Review shelf" after Finish is the only review. Tapping a bay there opens it on its own: crop it, page with Previous / Next (or a swipe), and go back with "Back to review" on the last bay or the close button.
- While AR tracking is limited the guidance shows "Hold steady Finding your place", with no distance and no direction. If tracking comes back in a moved world, the old bay positions are dropped and the guidance says "Lost your place Continue from where you are".
- 'arFlowShowsCapturePreview: true' (default 'false') brings back the per-bay preview. It also needs 'showCapturePreview', and the AR session keeps running under the preview. Forwarded on both platforms.

Rules:

- 'preset: 'multiBayAR' reads 'multiBay' and 'captureMode' from the top-level options; 'multiBayAR: {...}' reads them from the object. Giving either one in both places rejects with 'INVALID_ARGUMENT'.
- Any other option you set explicitly overrides that one field of the preset (e.g. 'showAlignmentGuidance: false', 'maxCaptures'). Options you omit keep the preset's value, not the plain 'openCamera' default.
- 'cameraMode: 'default' with the preset rejects with 'INVALID_ARGUMENT'.
- 'returnCaptureFileUri' stays whatever you set at 'init()'; the preset does not change it.
- Invalid 'captureMode', 'preset' or 'multiBayAR' values reject with 'INVALID_ARGUMENT' from the JS layer. The native bridges decode 'captureMode' leniently, as the native SDKs do (unknown = guided), so a value can never fail a whole session on the native side.
- 'getARCapabilities()' tells you, before opening the camera, whether AR capture will actually run (no 'init()' needed):

```
const { worldTrackingSupported, depthSupported, installRequired } =
  await Neurolabs.getARCapabilities();
// iOS: ARKit world tracking / LiDAR scene depth; installRequired is always false.
// Android: ARCore supported AND installed / ARCore Depth API; installRequired is
// true when Google Play Services for AR is missing or too old (the AR
// camera prompts for it, and falls back to the regular camera until then).
```

- On Android the plugin now declares ARCore ('com.google.ar:core') 1.56.0, the version the SDK is built against. If your app pins ARCore itself, pin 1.56.0 or newer.

6c. Capture and quality checks only (no boxes, no recognition)

For hosts that use the SDK camera, its image quality checks, review and the upload queue, and want neither detection boxes on screen nor on-device recognition. The detector still runs once it is on the device, for the shelf guidance; it only downloads on Wi-Fi or Ethernet.

Config.

```
await Neurolabs.init({
  apiKey: '<API_KEY>',
  autoSyncEnabled: true,
  wifiOnlyUploadsEnabled: false, // or the rep's own setting
  deletePhotosOnUploadSuccess: false, // the host deletes with deletePhoto (see the queue
  note below)
  detectorPrefetchPolicy: 'unmetered', // download the detector (~56 MB) only on Wi-Fi / E
  thernet
  recognition: false, // never build on-device recognition
});

await Neurolabs.openCamera({
  type: 'shelf',
  cameraMode: 'default',
  guidanceMode: 'guidance',
  validationPreset: 'lenient',
  sessionId,
  taskUUID,
  confidenceThreshold: 0.25,
  iouThreshold: 0.45,
  showCapturePreview: false,
  maxCaptures: 10,
  showAlignmentGuidance: true,
  enableValidation: true,
  liveQualityChecksEnabled: true,
  liveQualityTargetFps: 6,
  showDetections: false, // no boxes; detection keeps running for the guidan
ce
  showCapturedRegions: false,
  sendDetectionsMetadata: true,
  enablePreviewCropping: true,
  allowManualFinish: true,
  autoCloseAfterCapture: false,
});
```

- 'detectorPrefetchPolicy: 'unmetered'" needs native SDK 1.7.16. On a metered network (cellular, a metered or personal hotspot, or iOS Low Data Mode) the download waits and starts by itself once the device is on Wi-Fi or Ethernet, even mid-session. Until then 'getDetectorModelStatus()' and 'detectorModelStatusChanged' report '{ state: "not_downloaded", ready: false, reason: "waiting_for_unmetered_network" }'. That reason is not a failure. Opening the camera on cellular does not download either. To download anyway, call 'prefetchDetectorModel()'; an explicit call ignores the policy. A detector already on the device is used on any network. Omitting the key means "any", and the plugin applies it on every 'init'.
- 'recognition: false' stops the plugin from building the recognition bootstrap. There is no catalog snapshot or embedder download and no recognition network call, and 'captureResult' carries no 'recognitionMatches'. 'recognitionStatus' reports '{ status: "unavailable", reason: "disabled" }'. A re-init with 'false' retires a chain that an earlier init built.
- 'showDetections: false' hides the detection boxes, and 'showCapturedRegions: false' hides the captured-region overlay. Neither stops detection: once the detector is on the device, it runs every frame for the guidance below, and for 'captureResult.detections' and the detection metadata. This holds on both platforms; on the AR camera ('cameraMode: 'ar') 'showDetections: false' also hides the world-locked boxes.
- Android only: with both overlays off, an omitted 'validationPreset' or "minimal" makes the Android SDK use its 'IOS_PARITY' validation options. An explicit "default", "lenient" or "ios_parity" is not affected.
- 'validationPreset: 'lenient'" applies the native SDKs' lenient thresholds on both bridges. Plugin 1.6.6 mapped it to "default", so fewer captures are refused than with 1.6.6.

Quality checks before and after the detector is ready.

| When | Checks | |---|---| | From the first frame, detector or not (image-only) | blur, motion blur, exposure (dark / bright, exposure settling), glare, backlight, clipping, finger in frame, rotate the phone, perspective ("Face the shelf straight on", from the image and the device angle) | | Once the detector is on the device (it switches on mid-session, no reopen) | point at a shelf, move closer / step back / pull back, framing (frame more of the shelf, pan / tilt), row and product blur, level the camera, price tags (missing, blurry), cooler door |

While the detector is missing, the camera shows "Detector unavailable. You can still capture." (waiting for an unmetered network) or "Detector downloading". Capture, the shutter, review and the upload queue all work.

Readiness. 'loadingProgress' reaches 'step: "ready"' without the detector.

'waitUntilReady()' resolves on it. **Never wait for "model_ready"**: it never fires until the detector is on the device, and with "unmetered" that can be days for a rep who never uses Wi-Fi. None of the options above waits on the detector. To show whether detection is running, read 'getDetectorModelStatus()' or listen to 'detectorModelStatusChanged'.

Upgrade checklist from plugin 1.6.6. These are the behaviour changes since 1.6.6 that a capture, quality, review, queue and photo-API host meets, with the plugin version that brought each one.

1. **No bundled detector (1.7.13).** The detector downloads at runtime, so the first launch needs a network for detection. Capture and uploads work without it. With 'detectorPrefetchPolicy: 'unmetered'', that network must be Wi-Fi or Ethernet.

2. **'loadingProgress' (1.7.13).** Without the detector, 'loadingProgress' reaches ""ready"", never ""model_ready"". Replace any wait on ""model_ready"" with 'waitUntilReady()' or a check for ""ready"". 3. **'yoloModelVariant' (1.7.13).** 'all_in_one' is the only detector. 'nlb_model', 'yolo_v26', 'yolo_v11_seg', 'shelf_rows' and 'yolo11l_seg_sam' are still accepted, as deprecated aliases of it. An unknown value rejects 'init' with 'INVALID_ARGUMENT'.

4. **'customCameraTemplate' (1.7.10).** It must be an object. A JSON string ('JSON.stringify(template)'), number, array or boolean rejects 'openCamera' with 'INVALID_ARGUMENT'. Each field is read on its own, so a bad field falls back to its default and the rest still applies.

5. **'apiBaseUrl' (1.7.9).** The key is ignored. Uploads go to the operations API; use 'operationsBaseUrl' for staging.

6. **'operationsBaseUrl' (1.7.13).** It is applied on every 'init'. An 'init' without it resets the base to production. If you use staging, pass it on every 'init'.

7. **'totalSizeBytes' (1.7.13).** In 'getQueueStatus()' and 'queueStatusChanged', it counts only the image storage of pending, uploading and failed items. Completed items you keep ('deletePhotosOnUploadSuccess: false') are in 'totalCount' but not in 'totalSizeBytes'. 'getQueueDetails().totalSizeBytes' still sums every listed item.

8. **Boolean options (1.7.13).** Every boolean 'init' option takes 'true' / 'false', a number (0 is false), or ""true"" / ""false"", ""1"" / ""0"", ""yes"" / ""no"", ""on"" / ""off"". Any other value keeps the default. On Android, '1', ""yes"" and ""on"" used to be ignored.

9. **String options (1.7.12).** On Android, a 'null' or non-string option is treated as absent, as on iOS, and a non-string 'apiKey' rejects 'init'. 'getPhoto({ format })' rejects a non-string 'format', and 'setRetryPolicy(true)' rejects.

10. **iOS recognition frameworks (1.7.8).** The plugin always embeds the five recognition xcframeworks: 'RecognitionInterface', 'RecognitionEngine', 'RecognitionEngineQdrant', 'NLQdrantEdgeFFI' and 'RecognitionBootstrap'. They are linked even with 'recognition: false', and they are dynamic, so keep them embedded.

11. **Android Kotlin metadata (1.6.8).** Stock Cordova Android apps compile against the Kotlin 2.3 SDK AAR without changes; the plugin injects 'GradlePluginKotlinVersion 2.3.0'. Remove any workaround you added for "binary version of its metadata is 2.3.0". The toolchain needs minSdk 26, compileSdk 36, AGP 8.9.1 or later, Gradle 8.13 and JDK 17.

12. **Events while the camera is open (1.6.8; the rate limit is from 1.7.8).** Capture events ('captureQueued', 'captureResult', 'uploadSucceeded', 'uploadFailed', 'captureDeleted') are buffered while the native camera is on screen. They are delivered in order after 'cameraClosed'. 'queueStatusChanged', 'guidanceStateChanged' and 'detectorModelStatusChanged' collapse to the latest value, and 'queueStatusChanged' fires at most 4 times a second. If 'cameraClosed.droppedEventCount' is above 0, re-read the queue.

13. **'cameraClosed.error' (1.7.9).** A capture screen that failed ('CAMERA_PERMISSION_DENIED' on iOS; 'NOT_INITIALIZED', 'CAPTURE_RESULT_MISSING', 'CAPTURE_RESULT_UNREADABLE' or 'CAPTURE_FAILED' on Android) arrives as 'cameraClosed' with 'cancelled: false' and 'error: { code, message }'. It used to look like the rep cancelling.

14. **Portrait only (1.7.10).** Every capture reaches the server in portrait.

15. **Android ARCore (1.7.9).** The AAR is built against ARCore 1.56.0.

16. **Recognition at 'init' (1.7.1 / 1.7.2).** Since 1.7.1, 'init' builds the on-device recognition bootstrap whenever it has an operations key, which is always (the key falls back to 'apiKey'). The bootstrap asks the operations API whether the org has device recognition and, if it does, downloads the catalog snapshot and the embedder. Pass 'recognition: false' to keep the 1.6.6 behaviour: no recognition at all.

17. **Queue capacity with kept photos (unchanged since 1.6.6).** With 'deletePhotosOnUploadSuccess: false', an uploaded item stays in the queue as 'completed' until you 'deletePhoto' it. It counts toward 'maxQueueSize' (default 100) on both platforms. At the cap the next capture is not queued; the SDK raises 'QUEUE_FULL' and the plugin reports it as 'uploadFailed' with 'errorCode: "QUEUE_FULL"'. 10 sessions of 10 captures with no deletes fill the default queue. Delete each photo once you have it ('getPhoto', then 'deletePhoto'), raise

7. Post-Processing + Lifecycle Events

```
// Fires when the camera screen is dismissed (Done or Close pressed).
// captureCount is the number of photos queued for upload - NOT the photo data itself.
// To retrieve photo data, use getPhoto() with the captureId from captureQueued (see section 8).
Neurolabs.addListener('cameraClosed', ({ sessionId, cancelled, captureCount, message, droppedEventCount, error }) => {
  console.log('cameraClosed', sessionId, cancelled, captureCount, message);
  // The capture screen failed (e.g. camera permission denied, SDK not
  // initialized, unreadable session result). 'cancelled' is false; show
  // 'error.message' and branch on 'error.code'.
  if (error) showCaptureError(error.code, error.message);
  // 0 on every ordinary session. Non-zero means the deferred-event buffer
  // overran and the replay that follows is incomplete - re-read the queue.
  if (droppedEventCount > 0) Neurolabs.getQueueStatus().then(console.log);
});

// Fires once per photo taken, immediately after each capture is added to the upload queue.
// captureQueued events are buffered while the camera is open and always delivered after cameraClosed.
// Use the captureId here to retrieve the local photo via getPhoto().
// imageFileUri ('file:///...') is present ONLY when file-URI streaming is enabled
// via init({ returnCaptureFileUri: true }) - see 8.1.
Neurolabs.addListener('captureQueued', ({ captureId, queueItemId, sessionId, imageFileUri })
=> {
  console.log('queued', captureId, queueItemId, imageFileUri);
});

// Real-time capture guidance feedback (buffered + coalesced like captureQueued).
// Payload: { rule, tier, timestamp, byDegrees? } where rule is one of
// DISTANCE_TOO_CLOSE | DISTANCE_TOO_FAR | ANGLE_OFF | PANNING_TOO_FAST.
Neurolabs.addListener('guidanceStateChanged', (payload) => console.log('guidance', payload));

// Fires after the Neurolabs server has processed an uploaded photo and returned an analysis
// result.
// This is a server-side callback - it does NOT fire when the photo is taken locally.
// Payload: { captureId, sessionId, success, message, detectionCount, capturedRegionCount }
Neurolabs.addListener('captureResult', (payload) => {
  console.log('captureResult', payload);
});

Neurolabs.addListener('uploadSucceeded', (item) => console.log('uploaded', item));
Neurolabs.addListener('uploadFailed', (payload) => console.warn('uploadFailed', payload));

// QUEUE_FULL and STORAGE_LOW (native SDK 1.7.16: under 500 MB free) refuse a
// NEW capture before it is queued: its captureResult has success: false and
// no captureQueued follows. STORAGE_LOW is retryable once space is freed.
Neurolabs.addListener('uploadFailed', ({ errorCode }) => {
  if (errorCode === 'STORAGE_LOW') askRepToFreeSpace();
  else if (errorCode === 'QUEUE_FULL') Neurolabs.flushQueue();
});

// Kept uploaded photos reached 80% of maxRetainedPhotos (see 5).
// Payload: { count, bytes, oldestCreatedAt?, limit }
Neurolabs.addListener('retainedPhotosHigh', (payload) => console.warn('retainedPhotosHigh', payload));
Neurolabs.addListener('queueStatusChanged', (status) => console.log('queueStatusChanged', status));
```

8. Retrieving Photos Locally

Photos are stored in the device's app cache after capture. Use 'getPhoto()' to access them by 'captureId' (from 'captureQueued') or 'queueItemId'.

```
const capturedIds = [];  
  
Neurolabs.addListener('captureQueued', ({ captureId }) => {  
  capturedIds.push(captureId);  
});  
  
Neurolabs.addListener('cameraClosed', async ({ cancelled }) => {  
  if (cancelled) return;  
  for (const captureId of capturedIds) {  
    // format: 'fileUri' returns { uri: 'file://...' } - a temp path in the app cache  
    // format: 'base64' returns { base64: '...' } - requires allowBase64PhotoExport: true  
    in init()  
    const photo = await Neurolabs.getPhoto({ captureId, format: 'fileUri' });  
    console.log('photo uri', photo.uri);  
  }  
  capturedIds.length = 0;  
});
```

'deletePhoto()' accepts the same query shape ('captureId', 'queueItemId', or 'responseId') and removes the local file from the cache.

> In file-URI streaming mode (8.1) 'getPhoto({ format: 'base64' })' rejects with > 'UNSUPPORTED_FORMAT' - base64 export is forced off. Use 'format: 'fileUri'', or read the > 'imageFileUri' handed to you on 'captureQueued' directly.

8.1 Capture File-URI Streaming (v1.6.5, OOM fix)

Large multi-capture sessions (wide shelves, dozens of full-resolution JPEGs) could previously push peak memory high enough to get the app OOM-killed, because every captured image was held resident until the host consumed it.

Enable **file-URI streaming** to avoid this - an app-wide, opt-in toggle set once at 'init()':

```
await Neurolabs.init({ apiKey: '<API_KEY>', operationsApiKey: '<OPS_KEY>', returnCaptureFileUris: true });
```

Default is 'false' (unchanged behavior). When 'true':

- Each capture is written to a **stable, backup-excluded on-disk store** and handed to the host as 'imageFileUri' on the 'captureQueued' event, instead of being held in memory. Peak heap no longer grows with the number of captures.
- **base64 export is forced off**: 'getPhoto({ format: 'base64' })' rejects with 'UNSUPPORTED_FORMAT'. Read the 'imageFileUri' instead.
- The retained files **outlive the capture session** - they are kept until the host explicitly releases them, so you can stream/upload them after 'cameraClosed'.

The flag is honored on **both platforms**: Android bakes it into the init-level native configuration; iOS applies it to every capture session.

Releasing retained files

Because the files persist past capture-finish, the host **must** release them once it has finished streaming/uploading each 'imageFileUri', otherwise they linger until a 72-hour TTL sweep reclaims them. Two APIs:

```
const queued = [];  
Neurolabs.addListener('captureQueued', ({ captureId, sessionId, imageFileUri }) => {  
  queued.push({ captureId, imageFileUri });  
});  
  
Neurolabs.addListener('cameraClosed', async ({ sessionId, cancelled }) => {  
  if (cancelled) return;  
  for (const { captureId, imageFileUri } of queued) {  
    await uploadFromFileUri(imageFileUri);    // your host-side upload  
    await Neurolabs.releaseCapture(captureId);  // per-capture release  
  }  
  queued.length = 0;  
  
  // ...or release the whole session at once instead of per capture:  
  // await Neurolabs.acknowledgeCaptures(sessionId);  
});
```

- 'acknowledgeCaptures(sessionId)' - bulk release: deletes every retained file for the session. Missing/blank 'sessionId' rejects with 'INVALID_ARGUMENT'.
- 'releaseCapture(captureId)' - single-capture release, for hosts that upload incrementally. Missing/blank 'captureId' rejects with 'INVALID_ARGUMENT'.

Both are no-ops for sessions/captures that kept no file (legacy byte-return mode, or already released), and neither touches the SDK's own upload queue (a separate disk-backed store that streams its uploads independently).

9. Notes

- 'openCamera' is the custom native camera endpoint.
- Use the strict shelf payload above for full shelf guidance checks and auto-close after a validated save.
- 'liveQualityChecksEnabled=true' is required for pill/rotation/warning/error guidance behavior.
- Keep 'type: 'shelf', 'guidanceMode: 'strict', 'showDetections: false', and 'showCapturedRegions: false' for the custom-camera guidance UI path.
- 'guidanceMode: 'guidance' should only be used as a temporary fallback if a partner explicitly wants looser Android behavior than the parity recommendation.
- 'autoCloseAfterCapture=true' closes after Save from preview/review flow.
- 'Done' appears only in manual-finish mode: 'autoCloseAfterCapture=false' + 'allowManualFinish=true'.
- Use 'minCapturesBeforeDone' to require a minimum number of captures before 'Done' becomes active.
- If 'allowManualFinish=false', 'maxCaptures' acts as a hard limit and capture disables at the limit.

- 'captureQueued' events are buffered while the camera is open and always delivered after 'cameraClosed' - never during an active session. The buffer holds 512 entries; 'cameraClosed's 'droppedEventCount' reports how many it had to discard (0 unless a pathological session overran it).
- Keep base64 transport disabled unless explicitly needed.

Troubleshooting: "Unresolved reference" Kotlin errors on Android build

Symptoms like 'Unresolved reference 'setOperationsBaseUrl'', 'setMultiBaySubmitNotes', 'resourceName', or 'labelsResourceName' during the Android build mean the plugin source is being compiled against a **stale 'neurolabs-android-sdk.aar'** - an old AAR left in 'platforms/android/app/libs/' (which takes precedence) from a previous plugin version.

Fix:

1. Delete the stale AAR and its metadata:
'platforms/android/app/libs/neurolabs-android-sdk.aar' (+ '.meta.json')
2. Re-run 'cordova prepare android' so the plugin re-fetches the AAR matching the plugin's pinned native version.
3. Rebuild.

Since v1.6.3 the Android Gradle script fails the build early with an explicit version-mismatch message (instead of the confusing Kotlin errors) whenever the resolved AAR's version does not match the plugin's 'neurolabs.sdkVersions.android'.

Upgrading from a pre-1.6 plugin - queue durability

On pre-1.6 native SDKs the upload queue was wiped by the very bridge message that carries your API config. Sending 'NL_Config' / the 'api_config' action (which you do to set credentials + per-visit 'accountId'/ 'storeId'/ 'visitId') triggered a full queue clear on the native side - metadata **and** the stored JPEGs - so any not-yet-uploaded captures from a previous visit were deleted at the start of the next one, regardless of network.

- **Already-lost captures cannot be recovered.** They were removed from device storage by the reconfigure-wipe - even sessions that were never lost to timeouts were destroyed this way. There is nothing to migrate.
- **Upgrading stops the loss going forward.** From native v1.6.1+ (plugin bundling native 1.6.1) the config message no longer clears the queue; captures persist across reconfigures, backgrounding, and restarts. Only an explicit 'clearUploadQueue()' removes them.

Upgrade to a plugin bundling native **1.6.1** (1.6.3+ recommended) and set 'operationsApiKey' at init so the preserved queue can deliver.

On-Device Recognition + Tap-to-Product-Card (v1.7.1, Android)

Config-gated per account: your operations API key resolves your org's recognition config and vector dataset - enabling it is a backend flip, no app change. When active, 'captureResult' events carry 'recognitionMatches: { <detecionId>: { catalogItemId, similarity } }' (each detection object carries the correlating 'id'); resolve them for your card UI:

```
const details = await Neurolabs.getProductDetails(  
  Object.values(payload.recognitionMatches).map((m) => m.catalogItemId)  
);  
// details["<uuid>"] { canonicalUuid, name?, brand?, flavour?,  
//                      containerSize?, barcode?, thumbnailUrl? }
```

Absent keys are unknown products - render the raw UUID + similarity as the fallback.
Results are cached natively for 24 h. iOS parity for the recognition provider arrives
in a follow-up release; 'getProductDetails' works on both platforms today.